Signal Normalization and Quality Control Pipeline - Complete Guide

This comprehensive guide provides step-by-step instructions for setting up and running the signal normalization and quality control pipeline in WSL.

Overview

This pipeline provides:

- **Comprehensive quality control metrics** (FRiP, SNR, library complexity, etc.)
- **Signal normalization** (RPM, RPKM, TPM)
- **Multi-sample comparison** and aggregated reporting
- **Realistic test datasets** with different quality scenarios
- **Automated quality assessment** with pass/fail thresholds

Prerequisites

- **WSL Environment**: Ubuntu 20.04+ or similar Linux distribution
- **System Resources**: 8GB RAM, 20GB free disk space
- **Network**: Internet connection for dependencies
- **Time**: 45-90 minutes for complete setup and testing

Step 1: System Environment Setup

Install Essential Dependencies

bash

```bash
# Update system
sudo apt update && sudo apt upgrade -y

# Install build tools and dependencies
sudo apt install -y \
    build-essential \
    cmake \
    pkg-config \
    git \
    wget \
    curl \
    unzip \
    python3 \
    python3-pip \
    python3-venv \
    openjdk-11-jdk \
    zlib1g-dev \
    libbz2-dev \
    liblzma-dev \
    libcurl4-openssl-dev \
    libssl-dev \
    bc \
    bedtools \
    tabix

# Install additional Python packages
pip3 install --user numpy pandas matplotlib seaborn plotly
```

Install Rust

bash

```bash
# Install Rust
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh -s -- -y
source ~/.cargo/env

# Verify installation
rustc --version
cargo --version
```

Install Conda and Bioinformatics Tools

bash

```bash
# Install Miniconda
wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh -O miniconda.sh
bash miniconda.sh -b -p $HOME/miniconda3
export PATH="$HOME/miniconda3/bin:$PATH"
echo 'export PATH="$HOME/miniconda3/bin:$PATH"' >> ~/.bashrc
source ~/.bashrc

# Initialize conda
conda init bash
source ~/.bashrc

# Create bioinformatics environment
conda create -n biotools -y python=3.9
conda activate biotools

# Install bioinformatics tools
conda install -y -c bioconda \
    fastqc=0.11.9 \
    fastp=0.23.2 \
    bowtie2=2.5.1 \
    samtools=1.17 \
    bedtools=2.30.0 \
    deeptools=3.5.4 \
    ucsc-bedgraphtobigwig=377

# Install additional tools
pip install pysam biopython
```

Install Nextflow

bash

```bash
# Install Nextflow
wget -qO- https://get.nextflow.io | bash
sudo mv nextflow /usr/local/bin/
sudo chmod +x /usr/local/bin/nextflow

# Verify installation
nextflow -version
```

Step 2: Project Setup

Create Project Structure

bash

```bash
# Create main project directory
mkdir -p ~/signal_qc_pipeline
cd ~/signal_qc_pipeline

# Initialize Rust project
cargo init --name signal-normalization-qc

# Create additional directories
mkdir -p {src,scripts,docs,examples}
```

Set Up Project Files

Create the following files in your project directory:

1. Copy Cargo.toml

bash

```bash
# Copy the Cargo.toml content from the artifact
cat > Cargo.toml << 'EOF'
[package]
name = "signal-normalization-qc"
version = "0.3.0"
edition = "2021"
authors = ["Epigenomic QC Team"]
description = "Comprehensive signal normalization and quality control for epigenomic data"

[[bin]]
name = "rust_qc"
path = "src/main.rs"

[[bin]]
name = "rust_peak_caller"
path = "src/peak_caller.rs"

[[bin]]
name = "signal_normalizer"
path = "src/normalizer.rs"

[dependencies]
rust-htslib = "0.46"
rayon = "1.8"
serde = { version = "1.0", features = ["derive"] }
serde_json = "1.0"
clap = { version = "4.0", features = ["derive"] }
anyhow = "1.0"
log = "0.4"
env_logger = "0.10"
bio = "1.6"
itertools = "0.12"
statrs = "0.16"
dashmap = "5.5"
crossbeam = "0.8"
indicatif = "0.17"
chrono = { version = "0.4", features = ["serde"] }
flate2 = "1.0"
csv = "1.3"
regex = "1.10"
ndarray = "0.15"
plotly = { version = "0.8", features = ["kaleido"] }

[dev-dependencies]
tempfile = "3.0"
criterion = "0.5"
approx = "0.5"
```

```
[[bench]]
name = "qc_benchmarks"
harness = false

[profile.release]
opt-level = 3
lto = true
codegen-units = 1
panic = "abort"

[profile.dev]
opt-level = 1
EOF
```

2. Copy Source Files

bash

```
# Copy main.rs to src/main.rs (from artifact)
# Copy peak_caller.rs to src/peak_caller.rs (from artifact)
# Copy main.nf (from artifact)
# Copy generate_qc_test_data.py (from artifact)

# Make scripts executable
chmod +x generate_qc_test_data.py
```

Step 3: Build and Test Rust Components

Compile Rust Dependencies

bash

```
# Navigate to project directory
cd ~/signal_qc_pipeline

# Clean and build
cargo clean
cargo build --release

# This will take 5-15 minutes on first build
# Progress will be shown for dependency compilation

# Verify binaries
ls -la target/release/rust_*

# Test basic functionality
cargo run --release --bin rust_qc -- --help
cargo run --release --bin rust_peak_caller -- --help
```

Run Unit Tests

```bash
# Run Rust tests
cargo test --release

# Run with verbose output if needed
cargo test --release -- --nocapture
```

Step 4: Generate Comprehensive Test Dataset

Create QC Test Data with Multiple Quality Scenarios

```bash
# Generate test dataset with different quality scenarios
python3 generate_qc_test_data.py \
    --output-dir qc_test \
    --num-reads 120000 \
    --read-length 75 \
    --include-scenarios high_quality medium_quality low_quality \
    --peaks-per-mb 20.0 \
    --seed 42

# Verify generated files
ls -la qc_test/
ls -la qc_test/data/
ls -la qc_test/reference/
ls -la qc_test/regions/
```

Inspect Generated Test Data

bash

```bash
cd qc_test

# Check reference genome
head reference/genome.fa

# Check peak regions
head -10 reference/true_peaks.bed

# Check quality scenario descriptions
cat data/*_description.txt

# Verify FASTQ files
zcat data/high_quality.fastq.gz | head -8
zcat data/low_quality.fastq.gz | head -8

# Check file sizes
du -sh data/*.fastq.gz
```

Step 5: Test Individual Pipeline Components

Test Reference Indexing

bash

```bash
# Activate conda environment
conda activate biotools

# Build reference indices
echo "Building bowtie2 index..."
bowtie2-build reference/genome.fa reference/genome_index

echo "Building samtools index..."
samtools faidx reference/genome.fa

# Create chromosome sizes file
cut -f1,2 reference/genome.fa.fai > reference/chrom_sizes.txt

# Verify indices
ls -la reference/
```

Test Read Processing and Alignment

bash

```bash
# Test one high-quality sample
echo "Testing read processing pipeline..."

# Quality control and trimming
fastp \
    -i data/high_quality.fastq.gz \
    -o high_quality_trimmed.fastq.gz \
    --qualified_quality_phred 20 \
    --length_required 36 \
    --thread 4 \
    --json high_quality_fastp.json \
    --html high_quality_fastp.html

# Alignment
bowtie2 \
    -x reference/genome_index \
    -U high_quality_trimmed.fastq.gz \
    --threads 4 \
    --very-sensitive \
    | samtools view -bS -F 4 - \
    | samtools sort -@ 4 -o high_quality_aligned.bam -

# Index BAM
samtools index high_quality_aligned.bam

# Check alignment statistics
samtools flagstat high_quality_aligned.bam
samtools stats high_quality_aligned.bam | head -20
```

Test Peak Calling

bash

```bash
# Test simple peak caller
echo "Testing peak calling..."
cargo run --release --bin rust_peak_caller -- \
    high_quality_aligned.bam \
    --output high_quality_peaks.bed \
    --window-size 1000 \
    --min-coverage 3.0 \
    --threads 4

# Check results
head high_quality_peaks.bed
wc -l high_quality_peaks.bed
```

Test QC Analysis

bash

```bash
# Test comprehensive QC analysis
echo "Testing QC analysis..."
cargo run --release --bin rust_qc -- \
    --input high_quality_aligned.bam \
    --peaks high_quality_peaks.bed \
    --output high_quality_qc.json \
    --background-regions regions/background_regions.bed \
    --blacklist-regions regions/blacklist_regions.bed \
    --fragment-shift 75 \
    --extend-reads 150 \
    --bin-size 1000 \
    --threads 4 \
    --verbose

# Check QC results
cat high_quality_qc.json | jq .
```

Step 6: Run Complete Pipeline

Execute Full Nextflow Pipeline

bash

```bash
# Navigate to test directory
cd qc_test

# Run complete pipeline
echo "Running complete QC pipeline..."
nextflow run ../main.nf \
    -c nextflow.config \
    --input_dir data \
    --output_dir results \
    --reference_genome reference/genome.fa \
    --background_regions regions/background_regions.bed \
    --blacklist_regions regions/blacklist_regions.bed \
    --min_frip 0.01 \
    --min_snr 1.5 \
    --min_complexity 0.5 \
    --threads 4 \
    -with-report results/execution_report.html \
    -with-timeline results/timeline.html \
    -with-trace results/trace.txt

# Alternative: Use the generated run script
# ./run_qc_test.sh
```

Monitor Pipeline Progress

bash

```bash
# In another terminal, monitor progress
watch -n 10 'ls -la work/ | wc -l; echo "Processes:"; ls work/ | tail -5'

# Monitor system resources
htop

# Check log files
tail -f .nextflow.log
```

Step 7: Analyze and Validate Results

Examine Pipeline Outputs

bash

```bash
# Check results structure
ls -la results/
ls -la results/qc/metrics/
ls -la results/summary/
ls -la results/normalized/

# View individual QC results
cat results/qc/metrics/high_quality_qc_results.json | jq .
cat results/qc/metrics/low_quality_qc_results.json | jq .

# Check QC summaries
cat results/qc/metrics/*_qc_summary.txt
```

Validate Quality Classifications

bash

```bash
# Extract quality classifications
echo "Quality Classifications:"
for file in results/qc/metrics/*_qc_results.json; do
    sample=$(basename $file _qc_results.json)
    quality=$(jq -r '.overall_quality' $file)
    frip=$(jq -r '.frip_score' $file)
    echo "$sample: $quality (FRiP: $frip)"
done

# Expected results:
# high_quality: PASS (FRiP: >0.2)
# medium_quality: WARNING (FRiP: ~0.15)
# low_quality: FAIL (FRiP: <0.1)
```

Check Aggregate Reports

bash

```bash
# View aggregate QC report
# firefox results/summary/qc_aggregate_report.html

# Check metrics table
cat results/summary/qc_metrics_table.csv

# Review pass/fail summary
cat results/summary/qc_pass_fail_summary.
```